



Proyecto Integrador – **Introducción al Desarrollo de** **Software.**

Materia: Introducción al Desarrollo de Software.

Catedra: Lanzillotta.

Nombre del grupo: en_mi_compu_andaba.

Número del grupo: 20.

Tutor asignado: Martin.

Fecha de entrega: -Primer Entrega: 17/06/2026

-Segunda Entrega: 24/06/2026

Índice:

Resumen	Pag N°4
Introducción	Pag N°5
Solución Propuesta	Pag N°6
Hipótesis y supuestos	Pag N°6
Arquitectura General	Pag N°6
Base de Datos	Pag N°7
Parte Publica (cliente)	Pag N°7
Parte Privada (administrador)	Pag N°7
Gestión de Proyecto	Pag N°8
Tecnología utilizada	Pag N°9
Listado de Endpoints	Pag N°11
Conclusiones Finales	Pag N°13

Integrantes del Grupo:

	Nombre y Apellido	Email	Padrón
Integrante 1	Lucas Baéz	lbaez@fi.uba.ar	113183
Integrante 2	Tomas Ezequiel Rodríguez	terodriguez@fi.uba.ar	115932
Integrante 3	Tomas Accomasso	tacomasso@fi.uba.ar	115710
Integrante 4	Matias Losas	mlosas@fi.uba.ar	115193
Integrante 5	Francisco Venencio	fvenencio@fi.uba.ar	115551
Integrante 6	Lucio Cienfuegos	lcienfuegos@fi.uba.ar	115586
Integrante 7	Nicolas Wu	niwu@fi.uba.ar	114725
Integrante 8	Mateo Vaccarelli	mvaccarelli@fi.uba.ar	115843

Resumen:

El trabajo practico que desarrollamos es una aplicación web para una hamburguesería llamada Krusty Krab, inspirada y tematizada por Bob Esponja y fondo de bikini. Esta aplicación permite a los usuarios consultar virtualmente el menú y realizar reservas de mesas, mientras que al administrador le permite gestionar el local desde un panel privado. La solución que implementamos incluye:

- Uso de HTML, CSS y Javascript para el frontend.
- Uso de Flask para el backend.
- Uso de MySQL como base de datos.

El proyecto se organizó mediante un backlog de producto de 10 épicas, gestionado por un tablero de Kanban en GitHub.

Introducción:

El presente informe muestra el desarrollo de nuestra aplicación web para la hamburguesería “Krusty Krab”. El objetivo del proyecto fue diseñar e implementar un sistema que resuelva problemas reales de un negocio gastronómico, como la visualización del menú, la gestión de las reservas y la administración del local.

La solución fue pensada para dos tipos de usuarios distintos: el usuario, que podrá navegar por el menú, filtrando por restricciones alimenticias, revisar las reseñas que tiene el restaurante y realizar reservas de mesa/s sin necesidad de registrarse; mientras que el administrador gestiona los platos mostrados en el menú, como las mesas y las reservas, pudiendo consultar las estadísticas de su local en un panel privado.

El desarrollo de este proyecto se dividió en un grupo de épicas gestionadas por un tablero Kanban desde GitHub, haciendo un correcto uso de las ramas para hacer un buen trabajo en equipo. En este informe se detallan las decisiones tomadas, la arquitectura del proyecto, las tecnologías utilizadas y las conclusiones del equipo.

Estructura de Carpetas:

```
Proyecto-Final-Integrador/
+-- docker-compose.yml      <- Orquesta los 3 servicios Docker
+-- requirements.txt
+-- README
+-- docs/                   <- Documentación técnica
|
+-- backend/                <- API REST
|   +-- Dockerfile
|   +-- app.py              <- Punto de entrada Flask
|   +-- config.py           <- Variables de entorno
|   +-- requirements.txt
|   +-- db/                 <- Scripts SQL para inicialización
|       |   +-- 01_user.sql
|       |   +-- 02_schema.sql
|       |   +-- 03_setup.sql
|   +-- static/qrs/         <- QRs generados (uno por reserva)
|   +-- src/                <- Código fuente, dividido por DOMINIO
|       +-- utils.py        <- Helpers comunes (decoradores, errores)
|       +-- auth/           <- Login del admin
|           |   +-- routes.py
|           |   +-- service.py
|           |   +-- repository.py
|       +-- platos/         <- ABM de platos
|       +-- mesas/          <- ABM de mesas
|       +-- servicios/      <- Lectura de servicios extras
|       +-- reservas/       <- Reservas (más complejo)
|           |   +-- routes.py
|           |   +-- service.py
|           |   +-- repository.py
|           |   +-- qr_service.py    <- Genera el PNG del QR
|           |   +-- email_service.py <- Envía los mails
|       +-- resenas/        <- Reseñas
|
+-- frontend/               <- Aplicación web
|   +-- Dockerfile
|   +-- app.py              <- Punto de entrada Flask
|   +-- requirements.txt
|   +-- src/
|       |   +-- api_client.py    <- Cliente HTTP centralizado al backend
|       |   +-- utils.py         <- Decorador @require_admin
|   +-- templates/          <- Plantillas Jinja2 (HTML)
|       |   +-- base.html
|       |   +-- admin_base.html
|       |   +-- index.html
|       |   +-- menu.html
|       |   +-- reservas.html
|       |   +-- cancelar_reserva.html
|       |   +-- resena.html
|       |   +-- resenas.html
|       |   +-- admin_menu.html
|       |   +-- admin_mesas.html
|       |   +-- admin_reservas.html
|       |   +-- admin_servicios.html
|       |   +-- admin_qr.html
|       |   +-- ...
|   +-- static/             <- CSS, JS, imágenes
```

Solución Propuesta:

Antes de empezar a desarrollar la solución que realizamos, mostramos un listado de hipótesis y supuestos en los que nos basamos para este proyecto:

- El local tiene una única sucursal con capacidad fija.
- El horario de atención es fijo según día de la semana.
- Las reservas operan por slots fijos de ****1 hora****.
- El local cuenta con un conjunto fijo de mesas, cada una con su capacidad.
- Una mesa solo puede tener una reserva por slot.
- Al crear la reserva, el sistema asigna automáticamente la mesa más chica que cubra la cantidad de personas, optimizando la ocupación del local.
- La cantidad de personas por reserva tiene:
 - mínimo: 1 persona
 - máximo: capacidad de la mesa más grande del local.
- Cada reserva está asociada a un ****QR único****, utilizado como identificador y llave de acceso para:
 - cancelación
 - reseñas
- Las reseñas se publican libremente, pero solo pueden realizarlas usuarios cuya reserva fue marcada como ****consumida****.
- Las reseñas pueden ser dadas de baja por el administrador.
- Existe un rol de ****administrador****.

Arquitectura general:

La aplicación se basa en una arquitectura cliente-servidor. El backend fue desarrollado en Flask (Python), exponiendo un API REST que se comunica con una base de datos MySQL. El frontend fue construido en HTML, CSS y Javascript con un diseño responsive, consumiendo los endpoints definidos.

Base de Datos:

Se diseñaron 8 tablas relacionales para cubrir todos los parámetros que nuestra solución necesita:

- **Usuarios:** administradores del sistema con autenticación por email y contraseña hasheada.
- **Platos:** menú del local con nombre, código, descripción, categoría e imagen.
- **Restricciones alimenticias:** catálogo de restricciones (vegetariano, sin gluten, etc)
- **Platos restricciones:** tabla intermedia que relaciona muchos a muchos entre platos y restricciones.
- **Mesas:** mesas del local con su capacidad y su estado.
- **Reservas:** reservas realizadas por los clientes con estado Pendiente, Consumida, Cancelada o Expirada, token único para el QR y token para habilitar la reseña.
- **Reseñas:** comentarios y puntuación dejados por los clientes tras concluir si visita.
- **Servicios:** servicios adicionales ofrecidos por el local.

Parte pública (cliente):

El cliente accede al sitio sin necesidad de registrarse. Puede visualizar el menú completo con filtros por categoría y por restricciones alimenticias. Para hacer una reserva ingresa su nombre, email, fecha, hora y cantidad de personas; el sistema asigna una mesa disponible automáticamente y genera un código QR único. Una vez consumida la reserva, el cliente recibe un token que le permite dejar una reseña con comentario y puntuación.

Parte privada (administración):

El administrador accede mediante login con sesión. Desde el panel puede gestionar el menú (altas, bajas y modificaciones de platos con imágenes), administrar las reservas, marcar su estado y escanear el QR. También cuenta con un dashboard de estadísticas que muestra KPIs de ocupación, cancelaciones y actividad por periodo, calculados en tiempo real sobre la tabla de reservas.

Por último, la gestión del proyecto se organizó en 10 épicas dentro de un backlog de producto, gestionado mediante un tablero Kanban vinculado con el repo de GitHub. Se utilizaron ramas para optimizar el trabajo en equipo. El repositorio incluye scripts SQL separados: `schema.sql` para la creación de tablas y `setup.sql` para la carga de datos de prueba.

Tecnología Utilizada:

Backend:

-Con **Python** se desarrollaron todos los módulos del servidor: el manejo de reservas, la validación de la disponibilidad de las mesas, la generación de los tokens únicos y códigos QR, envío de emails de confirmación y cancelación, y el panel de administración.

-**Flask** fue el framework utilizado para construir la API REST del backend, definiendo los endpoints de platos, reservas, reseñas, servicios y administración. Cada endpoint recibe requests HTTP del frontend y responde en formato JSON.

-**MySQL** fue la base de datos utilizada para almacenar toda la información del sistema: los platos del menú con sus categorías y restricciones alimenticias, las mesas del local, las reservas con su estado y su token QR, las reseñas de los clientes y los datos de los administradores.

Frontend:

-**Flask y Jinja2** fueron utilizados para renderizar todas las paginas del sitio: menú, reservas, reseñas, contacto, preguntas frecuentes y nosotros. **Jinja2** permitió mostrar los datos del servidor dinámicamente en cada template, iterar sobre listas de platos y reseñas, y reutilizar componentes comunes como la navbar y el footer mediante herencia con el base.html.

-**HTML y CSS** fueron utilizados para construir y estilizar la estructura visual de todas las páginas del sitio, con una estética basada en el universo de Bob Esponja y un diseño responsive adaptable a distintos tamaños de pantalla.

-**Javascript** fue utilizado para agregar interactividad en algunas páginas, abriendo preguntas frecuentes en acordeón, ordenando las reseñas con mayor o menor puntaje y permitiendo ver la contraseña que se esta escribiendo al ingresar como administrador.

Gestión del proyecto:

GitHub fue utilizado para el control de versiones del proyecto, coordinando el trabajo en equipo mediante ramas (main, feature, etc) y un tablero Kanban organizativo.

Lista de Endpoints:

Todos los endpoints viven en el backend (http://localhost:5000). El frontend los consume con api_client.

Convenciones:

- [admin] = requiere admin logueado.
- Endpoints sin [admin] son públicos.

Autenticación:

Método	Endpoint	Descripción
POST	/auth/login	Login del admin con email + password
POST	/auth/logout [admin]	Cerrar sesión

Platos:

Método	Endpoint	Descripción
GET	/platos	Listar platos (filtro opcional por restricción)
GET	/platos/<id>	Ver un plato
POST	/platos [admin]	Crear un plato
PUT	/platos/<id> [admin]	Editar
DELETE	/platos/<id> [admin]	Baja Lógica

Mesas:

Método	Endpoint	Descripción
GET	/mesas [admin]	Listar mesas
GET	/mesas/<id> [admin]	Ver una mesa
POST	/mesas [admin]	Crear mesa
PUT	/mesas/<id> [admin]	Editar
DELETE	/mesas/<id> [admin]	Baja Lógica

Servicios:

Método	Endpoint	Descripción
GET	/servicios	Listar servicios
POST	/servicios [admin]	Crear
PUT	/servicios/<id> [admin]	Editar
DELETE	/servicios/<id> [admin]	Baja Lógica

Reservas:

Método	Endpoint	Descripción
GET	/disponibilidad?cantidad_personas=X&fecha=Y	Ver mesas disponibles
POST	/reservas	Crear reserva (genera QR + manda mail)
GET	/reservas/<token>	Ver reserva por token (público, lo usa el cliente desde el mail)
PUT	/reservas/<token>/cancelar	Cancelar (público)
GET	/admin/reservas [admin]	Listar todas las reservas
POST	/admin/reservas/consumir [admin]	Marcar como consumida (dispara mail de reseña)

Reseñas:

Método	Endpoint	Descripción
GET	/resenas	Listar reseñas publicadas
GET	/resenas/<id>	Ver una reseña
POST	/resenas	Crear (requiere token de reserva consumida)
DELETE	/admin/resenas/<id> [admin]	Baja lógica

Recursos Estáticos:

Método	Endpoint	Descripción
GET	/static/qrs/<token>.png	Descargar QR de una reserva

Conclusiones Finales:

El desarrollo del proyecto nos permitió, a todo el equipo, aplicar los contenidos vistos en la materia en un desafío real como lo es construir una aplicación web completa con backend, frontend y base de datos, usando las metodologías adecuadas para optimizar el proceso. Uno de los principales aprendizajes fue la separación entre el backend y el frontend como dos aplicaciones Flask distintas que se comunican mediante una API REST, obligándolos a dejar claro el flujo de tránsito de la información, desde el frontend, al backend y desde este a la base de datos.